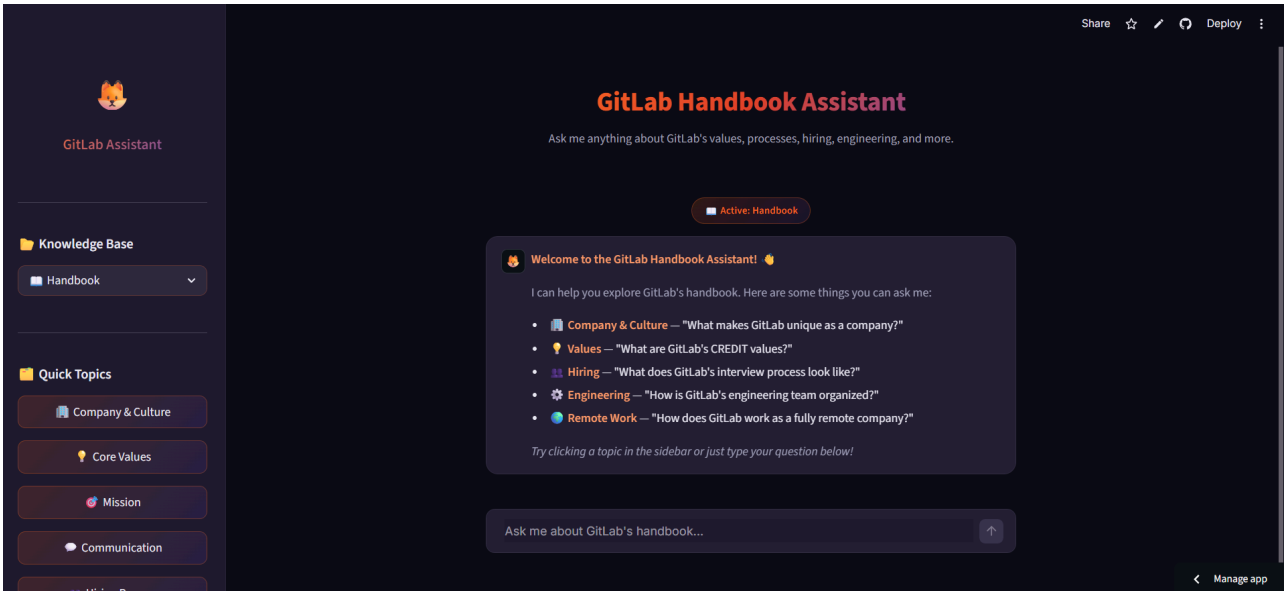
GitLab Assistant

GenAI Chatbot Project Documentation

A Retrieval-Augmented Generation (RAG) system for GitLab's Handbook & Direction

Submitted to	**Joveo AI**
Author	**Sanketh Katta**
Stack	**Python · Streamlit · FAISS**
Model	**OpenRouter Free Model**
Link	***https://joveo-sanketh-injdtmpv84ruuvhahkpt9x.streamlit.app/***
GitHub Link	***https://github.com/thunderstar123/Joveo-sanketh***



1. Project Overview

1. Project Introduction

The GitLab Assistant is an interactive AI-powered chatbot designed specifically for GitLab employees and candidates. Navigating the massive GitLab Handbook and Direction pages manually can be a daunting task. This project solves that by understanding natural language questions and providing relevant, grounded information directly from the source.

Following GitLab's "build in public" philosophy, this system is transparent and citable. It is strictly programmed to answer only from real, scraped GitLab content, ensuring that it never fabricates or hallucinates information.

1.1 What the chatbot can do

Core Capabilities

- **Handbook Expertise:** Answers questions regarding company culture, values, hiring, and remote work policies.
- **Direction Insight**: Provides details on product strategy, roadmaps, and release planning.
- **Contextual Awareness**: Maintains the flow of conversation across multiple follow-up questions.
- **Verified Citations**: Every response includes exact source URLs for user verification.
- **Reliability**: Gracefully refuses to answer off-topic questions to maintain accuracy.

1.2 Project brief alignment

Data Processing: Web scraping of both handbook.gitlab.com and about.gitlab.com/direction, with chunking and indexing.

Chatbot Implementation Hybrid RAG (FAISS semantic search + BM25 keyword search) with Jina cross-encoder reranking.

Frontend / UI Streamlit app with a premium dark theme, module selector, quick-topic buttons, and source citations.

Bonus Features PDF book export, Reciprocal Rank Fusion (RRF), Jina Reranker v3, per-module system prompts, and off-topic guardrails.

2. System Architecture

The system is built with a modular approach, split into four clearly separated layers: data ingestion, indexing, retrieval, and generation. This ensures that each part of the process has a single responsibility and can be updated or replaced without breaking the rest of the pipeline.

2.1 High-level data flow

```
Web Pages (GitLab)
     |
     v
scraper.py  — crawls, parses, chunks HTML pages
     |
     v   data/scraped/*.json
build_vectorstore.py  — embeds + indexes
     |
     v   data/faiss_index/<module>/
chatbot.py  — hybrid search + rerank + LLM
     |
     v
app.py (Streamlit)  — user-facing chat UI
```

2.2 Technology stack

Component	Technology	Purpose
Web Scraping	`requests + BeautifulSoup4`	Crawl & parse GitLab pages
Text Chunking	`LangChain RecursiveTextSplitter`	Split content semantically
Embedding Model	`all-MiniLM-L6-v2 (SBERT)`	Dense vector representation
Vector Store	`FAISS IndexFlatIP`	Semantic similarity search
Keyword Search	`BM25Okapi (rank_bm25)`	Exact-match term retrieval
Fusion / Merge	`Reciprocal Rank Fusion (RRF)`	Combine FAISS + BM25 scores
Reranker	`Jina Reranker v3 API`	Cross-encoder relevance boost
LLM	`OpenRouter`	Response generation
UI Framework	`Streamlit`	Chat interface & sidebar
PDF Export	`generate_pdf.py (custom)`	Export knowledge as PDF book

2.3 Module separation

One of the key architectural decisions was supporting two independent knowledge modules, Handbook and Direction, without code duplication. Each module has its own:

- Scraper configuration (domain, path prefix, output file name)
- FAISS + BM25 index stored in data/faiss_index/<module>/
- System prompt tailored to the domain's vocabulary and constraints

3. Step-by-Step Process

The pipeline runs as a sequence of four scripts. Steps 1–3 are one-time setup; Step 4 is the live application.

Step 1 Scrape GitLab Pages

Script: `scraper.py`

The scraper begins at each module's root URL and dynamically identifies section links from the navigation menu. It then crawls all valid internal pages in parallel using a `ThreadPoolExecutor` (batch size: 20).

Processing workflow for each page:

- Fetch HTML using a realistic user agent to prevent bot blocking
- Remove non-content elements (navigation, headers, footers, scripts)
- Extract structured text by grouping paragraphs under their nearest headings
- Split large sections using `RecursiveCharacterTextSplitter`
  - Chunk size: 2000 characters
  - Overlap: 300 characters
- Store each chunk as a JSON object with:
  - `ID`, `text`, `header`, `url`, `section`, `module`

```
python scraper.py --module handbook
python scraper.py --module directions
```

Step 2 Build the Vector Store

Script: `build_vectorstore.py`

This script processes scraped JSON chunks and creates two retrieval indices:

- **FAISS Index: Chunks** are embedded using *all-MiniLM-L6-v2* (384-dim, normalized) and stored in `IndexFlatIP` (cosine similarity via inner product).
- **BM25 Index: Text** is tokenized using a word-boundary regex and indexed with `BM25Okapi` for keyword-based retrieval.

Both indices, along with metadata (text, URL, section), are saved in: `data/faiss_index/<module>/`

```
python build_vectorstore.py --module handbook
python build_vectorstore.py --module directions
```

Step 3 RAG Query — Hybrid Search + Rerank

Module: `chatbot.py (HandbookChatbot class)`

When a user submits a question, the retrieval pipeline runs in four stages:

1. FAISS semantic search: The query is embedded and compared against all chunk vectors. Top-10 results are returned by cosine similarity.

2. BM25 keyword search: The query is tokenized and scored against the BM25 index. Top-10 keyword-match results are returned.

3. Reciprocal Rank Fusion (RRF): The two ranked lists are merged using the RRF formula (k=60) to produce a single ranked list of the best candidates.

4. Jina Reranker v3: The top candidates are re-scored by a cross-encoder model via the Jina API. The top-5 results after reranking are assembled as context.

The context and full conversation history are then sent to the OpenRouter API. The module-specific system prompt instructs the model to answer only from context, cite sources, use markdown formatting, and refuse off-topic questions.

Step 4 Streamlit Frontend

Script: `app.py + styles.py`

The Streamlit app provides a polished, single-page chat interface with:

- A sidebar dropdown to switch between Handbook and Direction knowledge bases
- Quick-topic buttons (12 per module) for one-click exploration
- A welcome message that adapts to the selected module
- Full conversation history rendered in a clean chat bubble layout
- Source citations appended to every answer
- A custom dark theme injected via st.markdown() with hand-written CSS

```
streamlit run app.py
```

4. Key Technical Decisions

4.1 Hybrid search
Semantic search can miss exact terms, while keyword search struggles with rephrased queries. Combining FAISS and BM25 using Reciprocal Rank Fusion balances both and improves retrieval quality without adding heavy computation.

4.2 Reranking with Jina
Initial FAISS results are fast but not always precise. A cross-encoder reranker refines a small set of candidates using deeper query-document understanding, improving accuracy while keeping latency low.

4.3 Embedding model
all-MiniLM-L6-v2 is lightweight, fast on CPU, and performs well for sentence similarity. Its compact vectors also make storage efficient in FAISS.

4.4 API choice
OpenRouter simplifies access to models through a single interface and supports fast, cost-effective experimentation for conversational systems.

4.5 Module-specific prompts
Each module has different intent and vocabulary. Separate system prompts help tailor responses, tone, and constraints more effectively without adding complexity during execution.

4.6 Chunking strategy
Text is split hierarchically from paragraphs to sentences to words to preserve meaning. A small overlap between chunks ensures continuity when information spans boundaries.

5. Bonus Features

5.1 PDF export
`generate_pdf.py` converts scraped module content into a structured, readable PDF, making it easy to use as an offline or printable reference.

5.2 Guardrails
System prompts enforce strict relevance. The assistant declines off-topic queries and ensures every response is grounded only in retrieved content, avoiding hallucinations.

5.3 Source citations
Each response includes source links, allowing users to verify information and explore the original content directly.

5.4 Conversation history
The full chat history is passed with each request, enabling the model to understand context across turns and handle follow-up queries naturally.

6. Project Structure

```
Joveo_AI/
├── app.py                  # Streamlit UI - module selector, chat, sidebar
├── chatbot.py              # RAG backend - hybrid search, rerank, LLM call
├── scraper.py              # Web crawler - Handbook & Direction modules
├── build_vectorstore.py    # Embedding + FAISS + BM25 index builder
├── generate_pdf.py         # PDF book export from scraped data
├── styles.py               # Custom Streamlit CSS theme
├── requirements.txt        # Python dependencies
├── .env                    # API keys (not committed to git)
├── .streamlit/
│   └── config.toml         # Streamlit theme config
└── data/
    ├── scraped/
    │   ├── handbook_chunks.json       # ~45 MB scraped Handbook
    │   └── directions_chunks.json     # ~1.4 MB scraped Direction
    └── faiss_index/
        ├── handbook/       # FAISS index + BM25 + metadata
        └── directions/     # FAISS index + BM25 + metadata
```

7. Setup & Running Locally

Prerequisites

- Python 3.9 or higher
- pip and a virtual environment (recommended)
- An OpenRouter API key (free tier available at openrouter.ai)
- A Jina API key (free tier available at jina.ai)

Installation

```
# 1. Clone the repository
git clone <repository-url>
cd Joveo_AI
```

```
# 2. Install dependencies
pip install -r requirements.txt
```

```
# 3. Set API keys in .env
OPENROUTER_API_KEY=sk-or-...
JINA_API_KEY=jina_...
```

```
# 4. Scrape data (one-time)
python scraper.py --module handbook
python scraper.py --module directions
```

```
# 5. Build indices (one-time)
python build_vectorstore.py
```

```
# 6. Launch the app
streamlit run app.py
```

8. Closing Remarks

This project presents a production-ready Retrieval-Augmented Generation pipeline built entirely on publicly available, free-tier infrastructure. The architecture is intentionally modular, with separate scraping configurations, independent indices, and tailored system prompts. This design makes it easy to extend the system to additional knowledge sources such as blog posts or release notes with minimal effort.

The retrieval strategy reflects a thoughtful balance between simplicity and performance. Instead of relying on a single embedding-based approach, the system combines FAISS, BM25, and reranking to handle both broad semantic queries and highly specific keyword-based searches. This added complexity is purposeful, as it leads to more reliable and relevant results across a wider range of user queries.

At its core, the system is built around user needs. Whether the user is an employee exploring internal processes or a candidate trying to understand the organization, the goal is to provide answers that are accurate, transparent, and grounded in original sources. By consistently linking responses back to their origins, the system not only delivers information but also builds trust and encourages deeper exploration without requiring users to manually navigate extensive documentation.

— End of Document —